



ARCHITECTURE RED FLAGS

& The Modern Backend Blueprint

La guía de supervivencia para arquitectos que no quieren que su sistema explote un domingo a la noche.

Tu arquitectura es un castillo de naipes.

Si estás acá es porque sabés que algo anda mal. La velocidad de entrega bajó y los bugs "inexplicables" subieron.

01 Efecto Dominó

Renombrás un campo en un DTO y se rompen 10 clases.

02 Lógica "Spaghetti"

El Controller valida, el Service guarda y la Entidad... solo existe.

03 La DB es tu API

Si cambiás una columna, rompés el contrato con el Front.

04 God Objects

Clases de 2000 líneas que nadie se atreve a tocar.

05 Test-fobia

"Es muy difícil testear esto", así que mejor no lo hacemos.

06 Vendor Lock-in

Tu lógica está tan pegada al framework que no podés migrar.

Dato crudo: El 70% del tiempo de desarrollo en backends legacy se pierde peleando contra el acoplamiento excesivo.

El DTO que quería serlo todo.

El error más común es usar la misma clase para recibir el Request, procesar el Service y devolver el Response.

EL ANTES (X)

```
// Un solo DTO para todo
public class UserDTO {
    private String password; // ¡Expuesto en la API!
    private List<Order> orders; // ¡Carga perezosa rota!
}
```

EL DESPUÉS (V)

```
public record CreateUserRequest(String email, String pass) {}
public record UserResponse(Long id, String email) {}
public class User { /* Lógica de negocio pura */ }
```

LA REGLA DE ORO: La duplicación explícita es más barata que el acoplamiento implícito. Siempre.

Services que hacen de todo menos lo que deben.

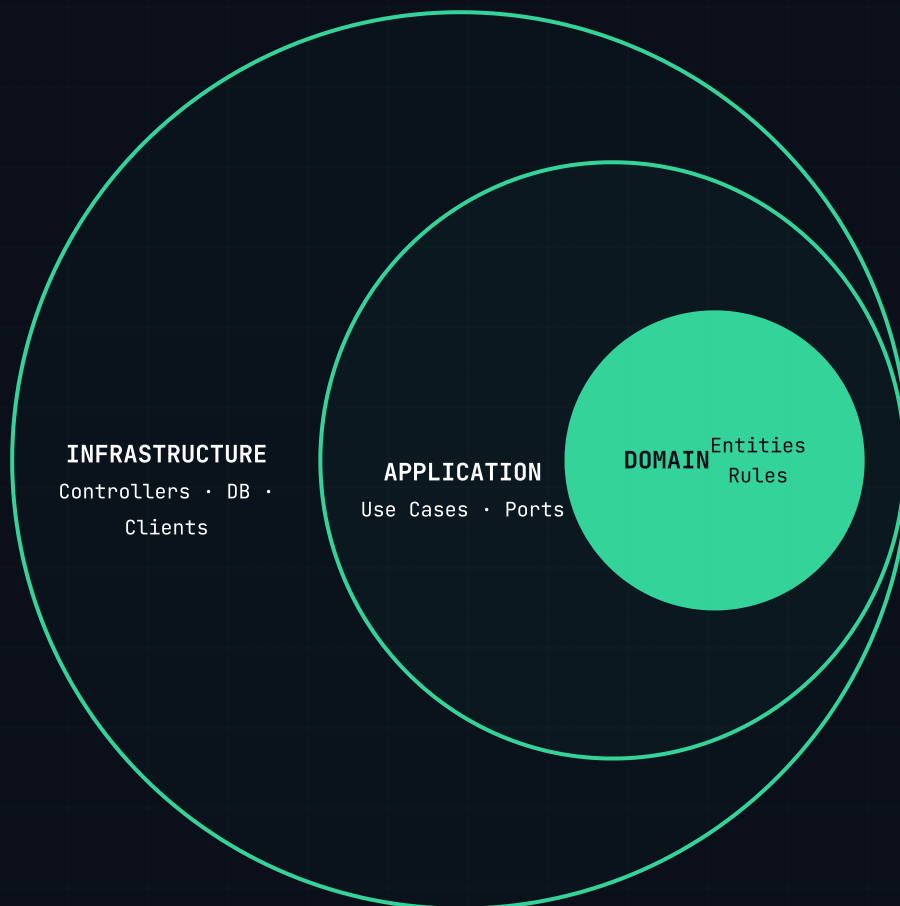
Si tu `UserService` manda mails, genera PDFs y valida tarjetas, tenés una bomba de tiempo.

Cómo arreglarlo en 3 pasos:

- **Extraé infraestructura:** Sacá el envío de mails a un `NotificationService`.
- **Aísla el dominio:** La lógica de "qué es un usuario premium" va en la Entidad, no en el Service.
- **Coordiná, no ejecutes:** El Service principal solo debe orquestar entre componentes.

// Métrica: Si no podés explicar qué hace una clase sin usar la palabra "Y", la cohesión es baja.

La Arquitectura Hexagonal para 2026.



Tu centro (el negocio) no sabe que existe la web ni la base de datos. Podés cambiar la DB y tu lógica sigue intacta. **Eso es libertad.**

Tu lógica de negocio es sagrada.

El código más valioso de tu empresa no es el que configura Hibernate, es el que dice cómo se calcula un descuento.

→ Entidades Ricas

La lógica de la entidad va DENTRO de la entidad. Nada de setters/getters vacíos.

→ Value Objects

Usá Email o Money en lugar de String. Validá al nacer, no al usar.

→ Contratos (Ports)

La aplicación define qué necesita. La infraestructura decide cómo lo da.

No publiques sin marcar estos puntos.

- ☐ **Observabilidad:** Correlation IDs y métricas Golden Signals.
- ☐ **Resiliencia:** Circuit Breakers y Timeouts explícitos.
- ☐ **Seguridad:** Sanitización de entradas y scopes mínimos.
- ☐ **Performance:** Índices DB y profiling de endpoints críticos.
- ☐ **Mantenibilidad:** Cobertura de tests para el Happy Path.

[!] LA CALIDAD NO ES NEGOCIABLE

¿Listo para subir el nivel de tu equipo?

Agendemos una llamada de 30 minutos para auditar tu arquitectura. Sin compromiso.

WEB

alafourca.dev

AGENDAR

cal.com/alafourcadev

"Los que más experiencia tenemos somos los que menos compartimos. Eso tiene que cambiar."

— ALEJANDRO LAFOURCADE